IY3660 – Applications of Cryptography

Week 5 Lecture
Hash functions

Elizabeth Quaglia

Information Security Group

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Quiz time!

In today's lecture

- Introducing hash functions
- Security of hash functions
- Constructions for hash functions
  - Merkle-Damgård paradigm
  - Constructions from block ciphers
- The MDx family: MD5, SHA-1, SHA-2
- SHA-3 and the sponge construction
- Applications of hash functions

Introducing hash functions

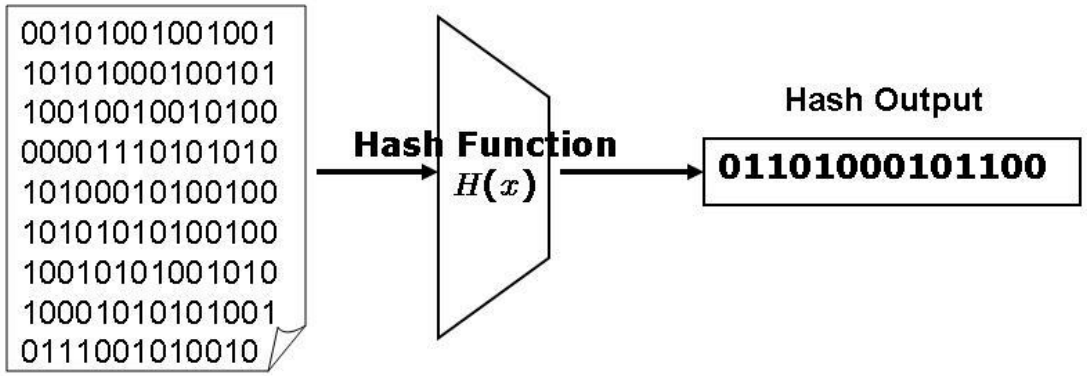
Cryptographic hash functions

An n-bit (cryptographic) hash function is an **efficiently computable** function

$$H\colon \{0,1\}^* \rightarrow \{0,1\}^n$$

mapping an arbitrary length input string to a fixed-length hash value (sometimes called a message digest)

- Hash functions produce a **digital fingerprint** of data
- Hash functions are **not** a keyed primitive



The Swiss Army knife of modern cryptography

- Message fingerprinting
  - File integrity checking, file sharing
- Signature schemes (hash-then-sign paradigm)
- Message authentication codes (e.g., HMAC)
- Key derivation
  - e.g., to derive keys from ‘raw data’ in Diffie-Hellman key exchange
- Password hashing
- Commitment schemes
- Pseudorandom number generators
- Stream ciphers (hash function in counter mode)
- Cryptocurrencies, e.g., Bitcoin (proof-of-work schemes)

Hash functions and the ROM

Random oracle model (ROM)

- Cryptographers like to think of hash functions as behaving like random functions
  - Given any input, output is a random n-bit string
- This is a strong idealisation
  - Is useful in formal security analysis
  - Not actually sound in practice

Standards for hash functions

NIST

- SHA-1 (until 2010)
- SHA-2 (includes SHA-224, SHA-256, SHA-384, SHA-512)
- SHA-3 (Keccak)

ISO/IEC (10118-3:2018)

- RIPEMD-128, RIPEMD-160, SHA-1, SHA-2 hashes, SHA-3, WHIRLPOOL, STREEBOG-512, STREEBOG-256, SM3

"Normal" hash functions are not good (for crypto)!

- Hash functions are widely used in computer science for non-cryptographic purposes
  - Hash tables, Bloom filters, error detection
- Non-cryptographic hash functions are weak!
  - Hash functions like CRC, Fletcher, Adler, Zobrist, Jenkins, MurmurHash, CityHash are absolutely useless for any cryptographic purposes!
- Example: Insecure hashing in WEP
  - The WLAN encryption protocol WEP is broken in part because CRC "hash" was used as cryptographic hash function
  - Problem: CRC is linear in the bits of the message

Non-cryptographic hash functions: digital checksums

- Simple form of hash function
- Often added to data to provide redundancy
- Typical use in communication channels:
  - Sender appends a short checksum to a message
  - Receiver can verify that the data has not been accidentally modified by re-computing the checksum
- Often computed using cyclic redundancy codes (CRCs)
- Can provide protection against accidental errors
  - Not against deliberate errors (e.g., active attacks)

Security of hash functions

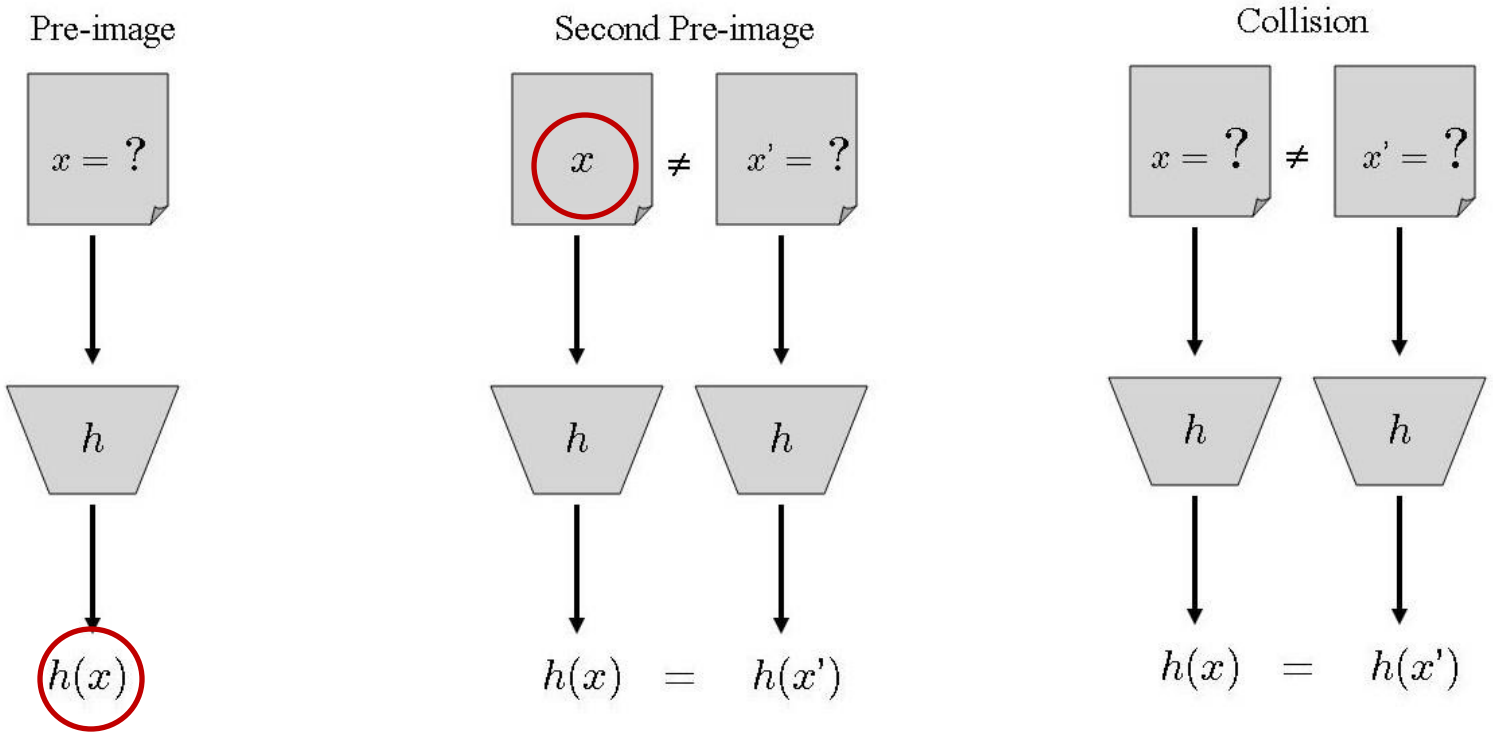
Cryptographic hash functions: security goals

- Pre-image resistance: given (randomly selected) y, it is infeasible to find $m \in \{0, 1\}^*$ such that $H(m) = y$
  - Preimage resistant hash functions are **hard** to invert
- Second pre-image resistance: given (randomly selected) m_1, it is infeasible to find $m_2 \neq m_1$ such that $H(m_1) = H(m_2)$
- Collision resistance: it is infeasible to find collisions, i.e., a pair of messages $m_1 \neq m_2$ such that $H(m_1) = H(m_2)$

NB: collisions must exist, because the input space of the hash function is much larger than the output space

- Collision resistance says that we can't find them efficiently

Cryptographic hash functions: security goals



: given, drawn at random.

Security goals for cryptographic hash functions

Secondary security goals

- Non-correlation: inputs and outputs of H should not be correlated
- Near-collision resistance: it is infeasible to find $m_1 \neq m_2$ such that $H(m_1) \approx H(m_2)$
  - (e.g. "≈" might mean hashes agree on most output bits)
- Partial pre-image resistance: given $H(m)$, it is infeasible to recover any partial information about m

NB these security goals are all stated informally;
their formalisation is beyond our scope in this module

Collision resistance implies second pre-image resistance

Theorem

Any hash function that is collision resistant is also second pre-image resistant

Proof:

- Assume H was collision resistant but not second pre-image resistant
- As H is not second pre-image-resistant one could pick a message m_1 and find a message $m_2 \neq m_1$ such that $H(m_1) = H(m_2)$
- This demonstrates a hash collision (m_1, m_2)
- This contradicts the assumed collision resistance of H

Relationships between security goals

- Collision resistance $\Rightarrow$ second pre-image resistance

- Second pre-image resistance $\not\Rightarrow$ collision resistance
  - (Exercise!)

What about the relationship between pre-image resistance and second pre-image resistance ?

Second pre-image resistance does **not** imply pre-image resistance!

Example

- Suppose G is a second pre-image resistant *n*-bit hash function

  Then define a new *(n+1)*-bit hash function H as follows:

  $H(m) = 1 \,||\, m$ if $\text{len}(m) = n$ bits;

  $H(m) = 0 \,||\, G(m)$ otherwise

- H is still second preimage-resistant (why?), but it is easy to find a pre-image (how?)

Second preimage resistance and pre-image resistance

- Preimage resistance does **not** imply second pre-image resistance
  - (Exercise!)

- Proofs rely on **artificial** constructions
  - Still shows the separation between these properties
- In **practice** collision resistant and second pre-image resistant hash functions are also pre-image resistant

Security property relationships: summary

- Collision resistance $\Rightarrow$ second pre-image resistance
- Second pre-image resistance $\not\Rightarrow$ pre-image resistance
- Second pre-image resistance $\not\Rightarrow$ collision resistance
- Pre-image resistance $\not\Rightarrow$ second pre-image resistance

- In theory, there is a separation between the main security properties
- In practice, collision resistant and second pre-image resistant hash functions are also pre-image resistant

Generic attacks on (1st / 2nd) pre-image resistance

- For any cryptographic hash function H: $\{0,1\}^* \rightarrow \{0, 1\}^n$
  - Given $y \in \{0, 1\}^n$, if H behaves like a random function, then by iterating over 2^n different messages m and for each one evaluating H(m), we should find m such that H(m) = y
  - Given m_1, if H behaves like a random function, by iterating over 2^n different messages m_2 and for each one evaluating H(m_2), we should find m_2 such that H(m_1) = H(m_2)

- We must ensure there are no better attacks on pre-image resistance and second pre-image resistance than these generic attacks
  - Output size n needs to be large to make these attacks infeasible

Generic attacks on collision resistance

- For any cryptographic hash function H: $\{0,1\}^* \rightarrow \{0, 1\}^n$
  - If H behaves like a random function, then by iterating over $2^{n/2}$ different messages m and for each one evaluating H(m), we should find m_1 and m_2 such that H(m_1) = H(m_2)

- Analysis relies on the birthday paradox
  - If elements are drawn at random from a set of size s, then after $\sqrt{s}$ trials, we expect a collision in the sampled elements with about 50% probability

- We must ensure there are no better attacks on collision resistance than this generic attack

Birthday attack probability analysis

- Suppose we have a set of s objects, and we sample at random from the set t times
  - Probability q that all the samples are different is:

$$q = 1 \cdot \frac{s-1}{s} \cdot \frac{s-2}{s} \cdot \frac{s-3}{s} \ldots \cdot \frac{s-t+1}{s}$$

  - Hence probability p of a collision is:

$$p = 1 - q = 1 - \frac{(s-1)(s-2)\ldots(s-t+1)}{s^{t-1}}$$

  - For s large, then if $t \approx \sqrt{s}$, this probability p is about 0.5

- For example, if $s = 365$, then $t \approx 23$ (the birthday paradox)

Let’s try it

For example, if $s = 365$, then $t \approx 23$ (birthday paradox)

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec

Birthday attack probability analysis

- An n-bit hash function offers only $\frac{n}{2}$ bits of security against the generic collision attack
- We need hash functions with 256-bit outputs to achieve 128-bit security level (that is, security against an attacker with storage/computation "resources" of size 2^{128})
- SHA-1 has an output size of 160 bits, so can only reach 80-bit security level against generic collision attack

Constructions for hash functions

Hash function design

- Two main design paradigms:
  - Based on block ciphers
  - Permutation-based ("sponges")

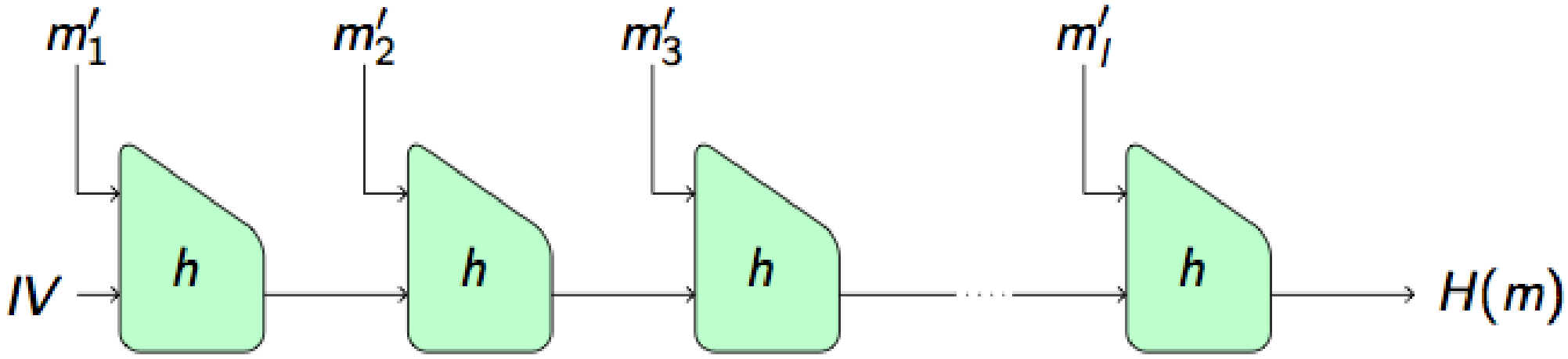
- A difficulty is that a hash function must be able to process arbitrary length messages
  - Leads naturally to the idea of an iterative design

Merkle-Damgård iterated hashing

- Constructs a hash function from a compression function

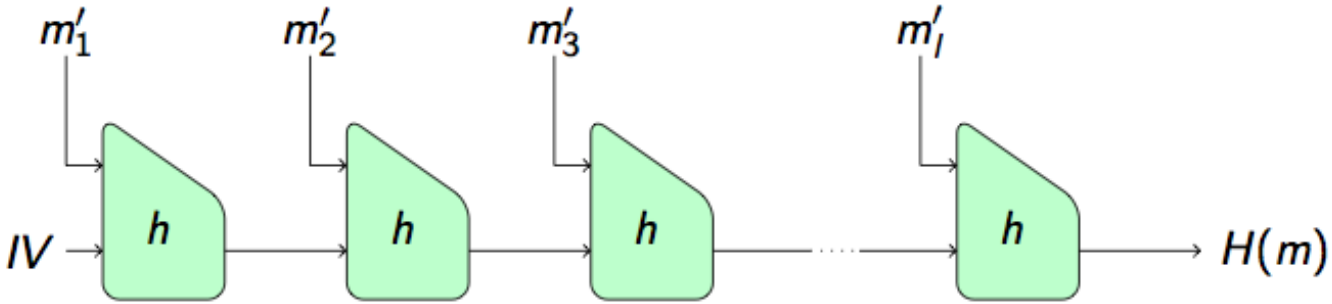
$$h : \{0,1\}^{n+k} \rightarrow \{0,1\}^n$$

- Prominent instantiations: MD5, SHA-1, SHA-2, WHIRLPOOL



Merkle-Damgård hashing of message m

- Let k and n be integers, and $IV \in \{0,1\}^n$ be a fixed constant
- Let padding scheme pad$(\cdot)$ be such that length of $m' = \text{pad}(m)$ is a multiple of k in bits
- Break-up m' into blocks $m_1{}', \ldots, m_l'$ each of length k
- Process blocks as shown **below** to compute hash value $H(m)$
- Each step processes a chunk of k input bits, output is n bits



Security of the Merkle-Damgård construction

Theorem

Suppose pad(m) transforms m into

$$m' = m \,||\, 1\, 0^t \,||[\text{len}(m)_k]$$

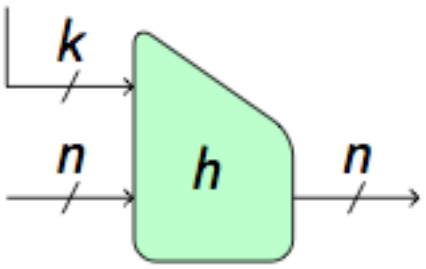
where $0 \leq t < k$ is minimal such that k divides len(m') and "[len(m)$_k$]" denotes the k-bit representation of the length of m

Then, **if compression function h is collision-resistant, so is H**

- **NB**: the converse is not true: H may still be collision-resistant even if h is not
- i.e., there may be collisions in h which do not translate into collisions for H

Construction of compression functions from block ciphers

The Merkle-Damgård iterated hash function result means that we've now reduced our problem of constructing a **good hash function** to that of constructing a **good compression function h**

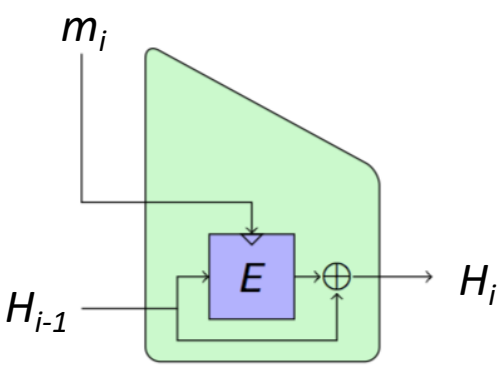


- Interface requirement: $h : \{0,1\}^{n+k} \rightarrow \{0,1\}^n$
  - h maps $n + k$ bits to n bits
- Security requirements: collision resistant, one-way,…

Can we re-use existing components, like block ciphers?

Construction of compression functions from block ciphers

- Examples: Davies-Meyer, Matyas-Meyer-Oseas, Miyaguchi-Preneel
- Most designs (e.g., the Davies-Meyer construction) input message blocks as the **key** of the block cipher
  - Some may need additional key processing to map input to key space
  - Implies that rekeying of block cipher needs to be fast
  - Hash output size = block cipher block size



Davies-Meyer

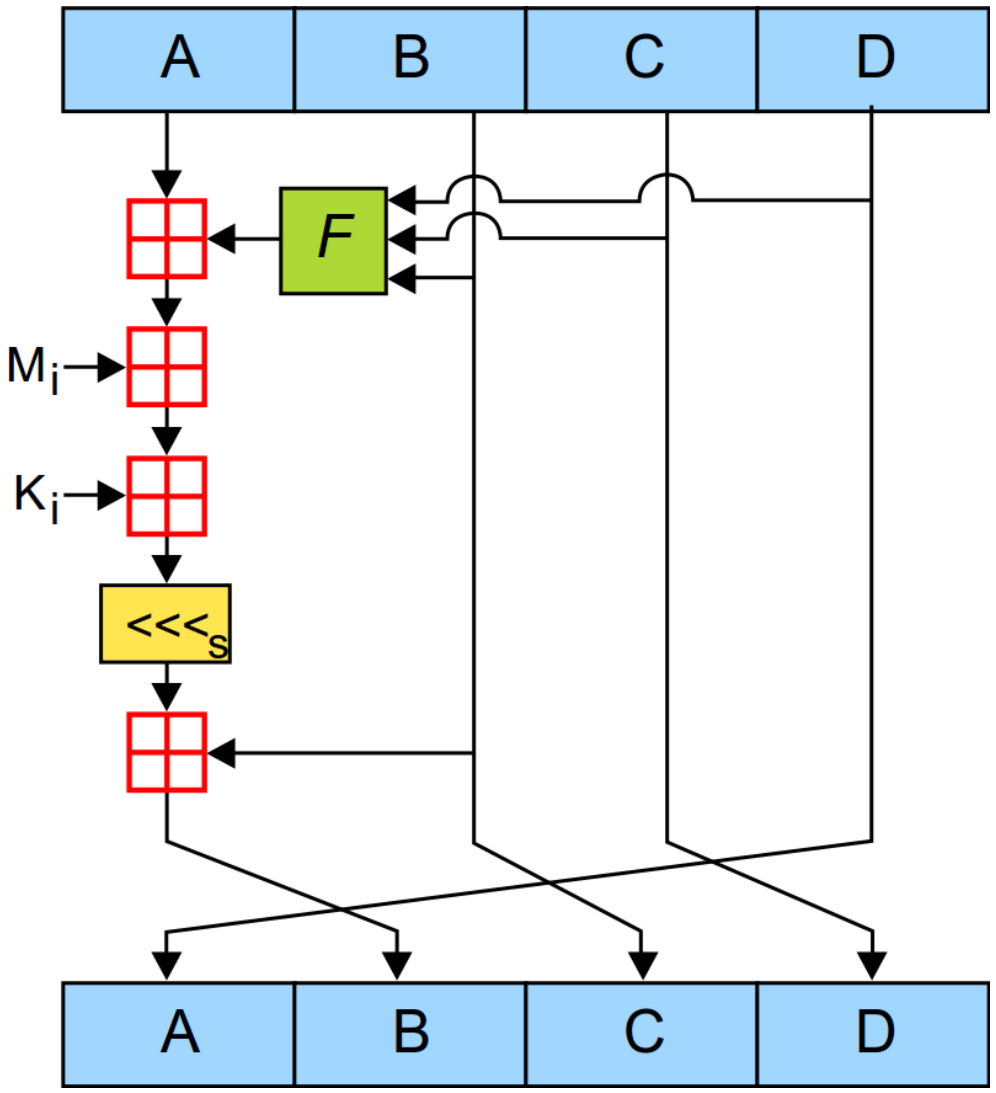
The MDx family

The MDx-family

- Derived from an early design from Ron Rivest in the early 1990s
- MD4, MD5, RIPEMD, SHA-1, SHA-2
- Very efficient
- Particularly suited to software implementations in 32-bit machines
- NIST and ISO/IEC 10118-3 standards
- Despite wide range of choices and security issues, SHA-1 remains deployed today, e.g., IPSec, SSL/TLS, SSH
  - However, this has now started to change…

MD5

- Merkle-Damgård hash with output size n equal to 128
- Compression function uses (implicitly) a special block cipher in Davis-Meyer mode, $k = 512$.
- Recall DM: message blocks → key; chaining variable → message
- Block cipher has a round function operating on 4 x 32-bit words
- Round function iterated 64 times
- Use mod 2^{32} additions, bit shifts, other simple bit operations (F)



"MD5" by Surachit - self-made SVG, based on [1] by User:Matt Crypto. Licensed under CC BY-SA 3.0 via Commons - https://commons.wikimedia.org/wiki/File:MD5.svg#/media/File:MD5.svg

Insecurity of MD5

- **1991:** MD5 designed to replace weak MD4
- **1993, 1996:** researchers identify initial flaws in MD5, recommend replacement
- **2004:** researchers show that MD5 is not collision-resistant; proof by example: everybody can reproduce the collisions!
- **2005:** researchers generate X.509 certificates with different keys but matching MD5 hash; very practical and serious security threat
- **2008:** researchers obtain fake MD5-based certificate from Verisign
- **2012:** flame malware uses fake certificate issued by Microsoft, exploiting MD5 collisions

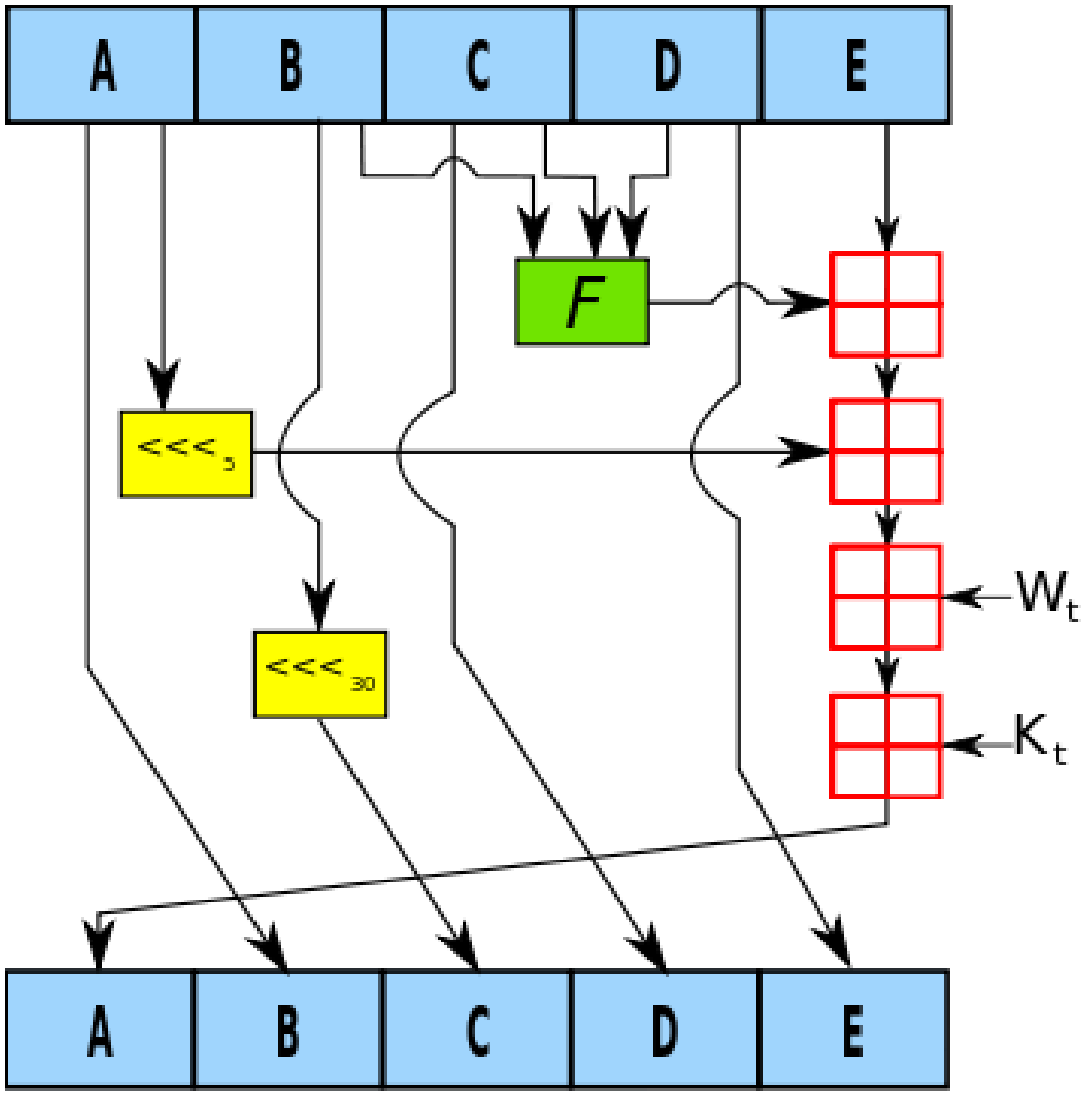
Why does it take so long to phase out a broken cryptographic algorithm?

Possible answers:

- Hard to do once deployed
- Backwards compatibility may be desirable
- Practitioners seem to require practical demonstration of attack (less conservative than cryptographers)
- Lack of understanding of how attacks evolve outside narrow circle of crypto experts?
  - Attacks really only do get stronger!
- Attacks only on collision-resistance, do not break all security properties (e.g., one wayness), so MD5 still ok in some applications, leading to confusion?

SHA-1

- Merkle-Damgård hash with output size *n* equal to 160 (20-byte output)
- Should provide 2^{80} security against collision attacks
- Compression function uses (implicitly) a special block cipher in Davis-Meyer mode, *k* = 512
- Recall DM: message blocks → key; chaining variable → message
- Block cipher has a round function operating on 5 x 32-bit words
- Round function iterated 80 times
- Use mod 2^{32} additions, bit shifts, other simple bit operations (F)



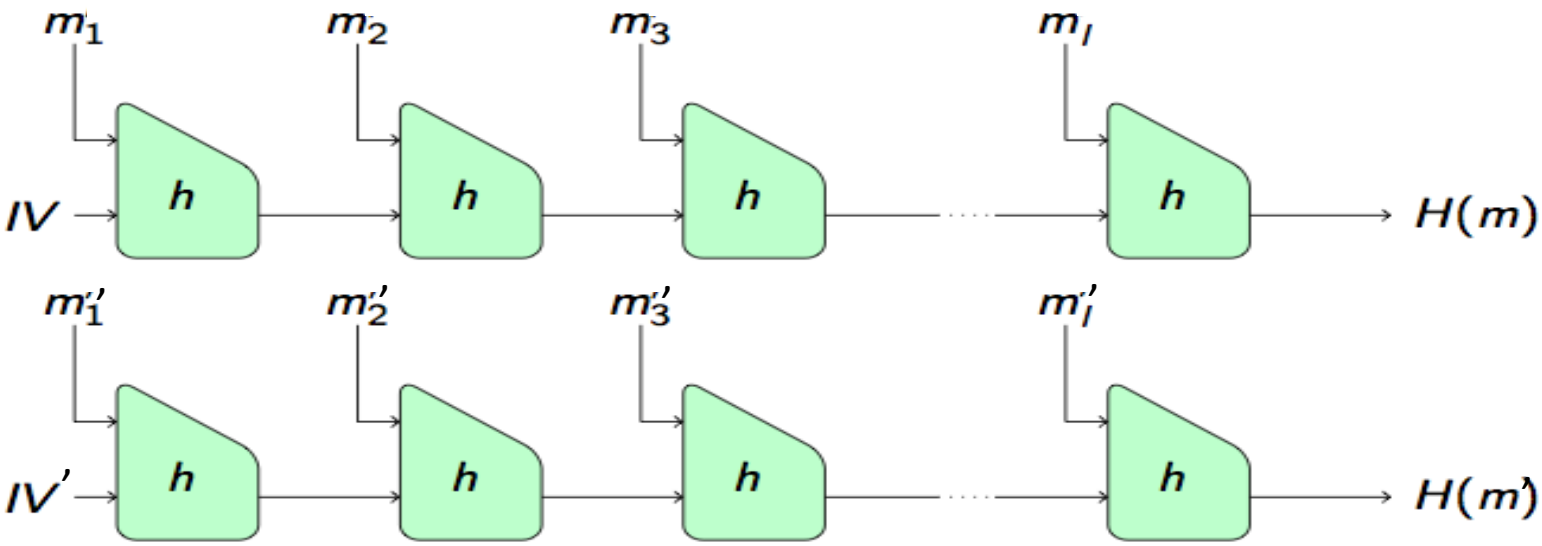
"SHA-1". Licensed under CC BY-SA 2.5 via Commons – https://commons.wikimedia.org/wiki/File:SHA-1.svg#/media/File:SHA-1.svg

Compressed timeline of security of SHA-1

- **1995**: Initial (non-public) analysis of SHA-0 led to a tweak to its design to produce SHA-1
- **2005**: First attacks on SHA-1 (Wang *et al*): 2^{69} compression functions evaluations (not implemented)
- **2005-2015**: Lots more analysis, collisions in around 2^{61} compression function evaluations; no attacks on full hash function or compression function implemented, but collisions for up to 77 rounds out of 80
- **2012:** NIST starts to retire SHA-1 http://csrc.nist.gov/groups/ST/hash/policy.html
- **Late 2014:** Google starts to penalise sites using SHA-1 certificates. (See also Microsoft's policy for code signing: http://social.technet.microsoft.com/wiki/contents/articles/32288.windows-enforcement-of-authenticode-code-signing-and-timestamping.aspx)
- **October 2015**: first **freestart** collision attack announced on full SHA-1
- **February 2017**: concrete collision on full SHA-1! https://shattered.it/
- **May 2019**: chosen-prefix collision attack on SHA-1 with complexity between $2^{66.9}$ and $2^{69.4}$
- **Jan 2020**: first practical implementation of the chosen-prefix collision attack, allowing a PGP/GnuPG impersonation attack

The SHAppening (Stevens, Karpman, Peyrin 2015)

- **Freestart** collision is significantly weaker attack than full collision
- Attacker gets to choose two different IVs in Merkle-Damgård construction
- In practice the IVs are fixed to a single value!
- Attack used 64 GPUs and took a few days



The SHAppening (Stevens, Karpman, Peyrin 2015)

A freestart collision example for SHA-1

The following two input IV/message pairs give the same output value after applying the SHA-1 compression function:

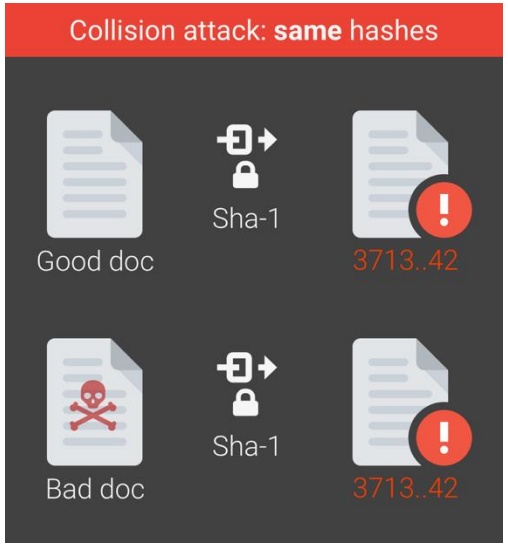
	Input 1
IV1	50 6b 01 78 ff 6d 18 90 20 22 91 fd 3a de 38 71 b2 c6 65 ea
M1	9d 44 38 28 a5 ea 3d f0 86 ea a0 fa 77 83 a7 36 33 24 48 4d af 70 2a aa a3 da b6 79 d8 a6 9e 2d 54 38 20 ed a7 ff fb 52 d3 ff 49 3f c3 ff 55 1e fb ff d9 7f 55 fe ee f2 08 5a f3 12 08 86 88 a9
SHA1_compression_function (**IV1**,**M1**)	f0 20 48 6f 07 1b f1 10 53 54 7a 86 f4 a7 15 3b 3c 95 0f 4b

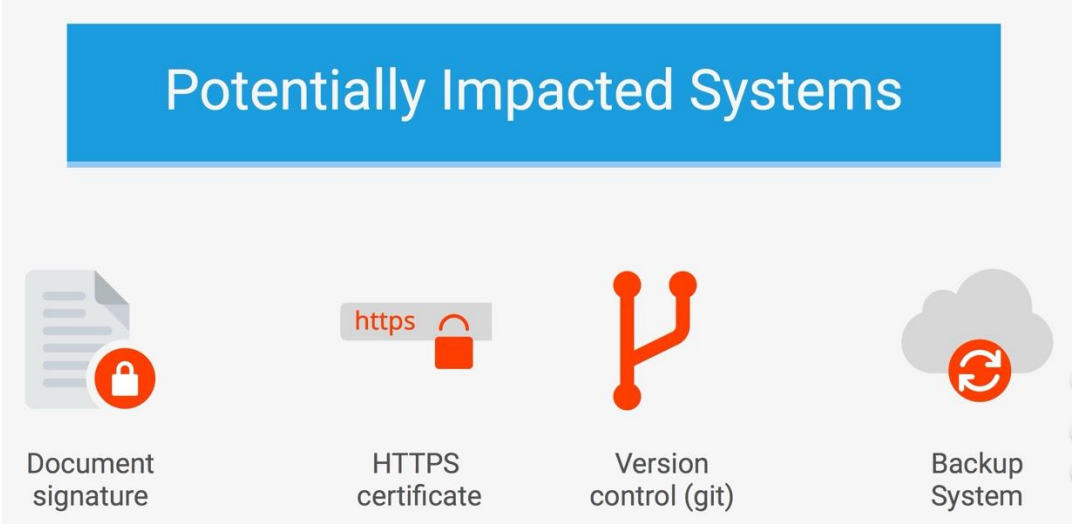
	Input 2
IV2	50 6b 01 78 ff 6d 18 91 a0 22 91 fd 3a de 38 71 b2 c6 65 ea
M2	3f 44 38 38 81 ea 3d ec a0 ea a0 ee 51 83 a7 2c 33 24 48 5d ab 70 2a b6 6f da b6 6d d4 a6 9e 2f 94 38 20 fd 13 ff fb 4e ef ff 49 3b 7f ff 55 04 db ff d9 6f 71 fe ee ee e4 5a f3 06 04 86 88 ab
SHA1_compression_function (**IV2**,**M2**)	f0 20 48 6f 07 1b f1 10 53 54 7a 86 f4 a7 15 3b 3c 95 0f 4b

source: https://sites.google.com/site/itstheshappening/ (now dead)

the SHA1 TERED attack

- **Full collision** on SHA-1 by Stevens, Karpman, Bursztein, Albertini & Markov, Feb. 2017
- Required 2^{63} SHA-1 computations, ≈ 6500 CPU-years + 100 GPU-years
- **Estimated cost of full collision attack on SHA-1: 100k-500k USD**





from https://shattered.it

Chosen-prefix collision attacks

- A **chosen-prefix collision** attack is a stronger variant of a collision attack:
  - Message prefixes P and P' are given as challenge to the adversary
  - Adversary's goal is to compute two messages M and M' such that

$$H\,(P \,||\, M) \;=\; H\,(P' \,||\, M')$$

- Significantly harder to produce than (identical-prefix) collisions, but the practical impact is much higher
- Collision attacks are typically hard to turn into a break of concrete protocols, because adversary may not control the colliding messages
- Chosen-prefix collisions are more powerful: P and P' can be *meaningful* messages
  - Can lead to breaking digital certificates (by creating a rogue CA) and internet protocols (e.g. TLS)

Chosen-prefix collisions for SHA-1

Chosen-prefix collision for SHA-1
(Leurent and Peyrin, May 2019 & Jan 2020)

- 2019 : turning SHA-1 collisions into chosen-prefix collisions
- 2020 : practical implementation of the attack, allowing PGP/GnuPG impersonation
- complexity of the attack: $2^{67.1}$ down to $2^{63.4}$
  (SHA-1 computations, costing around $45K)
- **Main message:** SHA-1 should no longer be used in applications requiring collision-resistance

SHA-1 is a Shambles

We have computed the very first **chosen-prefix collision for SHA-1**. In a nutshell, this means a complete and practical break of the SHA-1 hash function, with dangerous practical implications if you are still using this hash function. To put it in another way: all attacks that are practical on MD5 are now also practical on SHA-1.

Check our paper **here** for more details. Slides from RWC are also available.

Our Contributions

Complexity Improvements

We have significantly improved the complexity of SHA-1 attacks, with a speedup factor around 10. More precisely, we have reduced the cost of a collision attack from $2^{64.7}$ to $2^{61.2}$, and the cost of a chosen-prefix collision attack from $2^{67.1}$ to $2^{63.4}$ (on a GTX 970 GPU).

Record Computation

We implemented the entire chosen-prefix collision attack with those improvements. This attack is extremely technical, contains many details, various steps, and requires a lot of engineering work. In order to perform this computation with a small academic budget, we rented cheap gaming or mining GPUs from GPUserversrental, rather that the datacenter-grade hardware used by big cloud providers. We have successfully run the computation during two months last summer, using 900 GPUs (Nvidia GTX 1060).

As a side result, this shows that it now costs less than 100k USD to break cryptography with a security level of 64 bits (i.e. to compute 2^{64} operations of symmetric cryptography).

https://eprint.iacr.org/2019/459 and https://eprint.iacr.org/2020/014

SHA-2 family

- SHA-256: similar in design to SHA-1, but now using 256-bit "block cipher" (SHACAL-2) in Davies-Meyer mode to build compression function
  - $n = 256, k = 512$
  - 64 rounds, operating on 8 x 32-bit words using simple operations
  - Slower than SHA-1 (150 MB/s on modern CPU instead of 200 MB/s)
  - No attack faster than generic birthday attack (2^{128}) currently known

- SHA-224, SHA-384 and SHA-512 also available
  - Less widely deployed/used

SHA-3 and the sponge construction

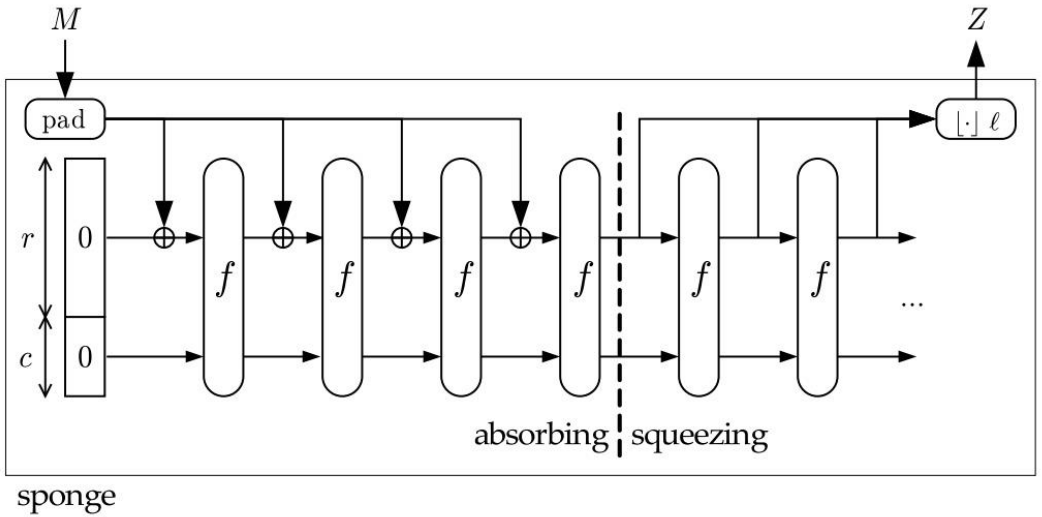
SHA-3

- In 2007, NIST launched an AES-style competition to design new hash functions
- The closure of the call period was at the end of 2008
- 63 submissions were received, 51 were selected for first round
- Two conferences to discuss candidates were held
- 14 semi-finalists were selected for further analysis
- 5 finalists were selected at the end of 2010
- Keccak was announced as winner in 2012
- NIST standard finally published in 2015 (FIPS 202)

SHA-3

- Designed by researchers from STMicroelectronics and NXP
  - Including Joan Daemen (AES co-designer)
- Based on a new design approach: a “sponge” construction
  - Unrelated to MD4, MD5, SHA-1, SHA-2 families
- Uses a “giant bit permutation” as a key component of the construction
- Variable length output
- Variable throughput, allowing efficiency/security trade-offs
- Chosen for elegance, security margins, performance and flexibility
- Intended to complement, not replace, SHA-2

Permutation-based construction: sponge



Sponge construction has two phases: absorbing and squeezing

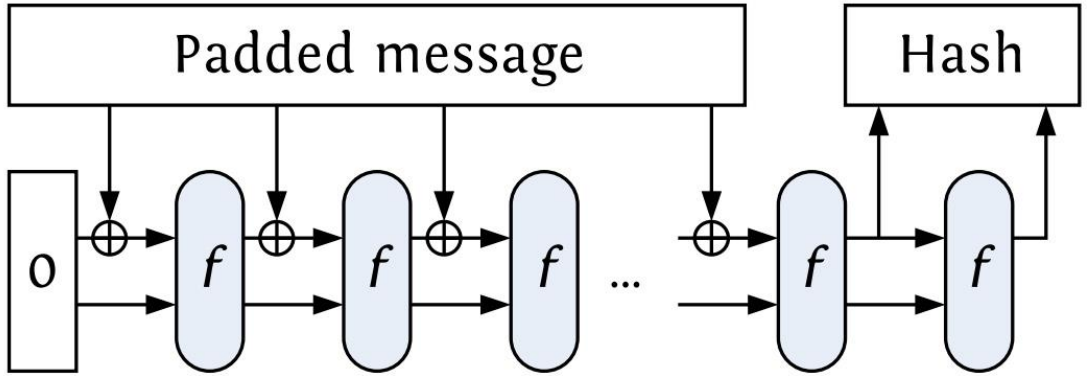
f: a b-bit permutation with $b = r + c$

- r: rate; c: capacity
- efficiency: processes r bits per call to f
- security: provably resists generic attacks up to $2^{c/2}$
- flexibility in trading rate for capacity or vice versa

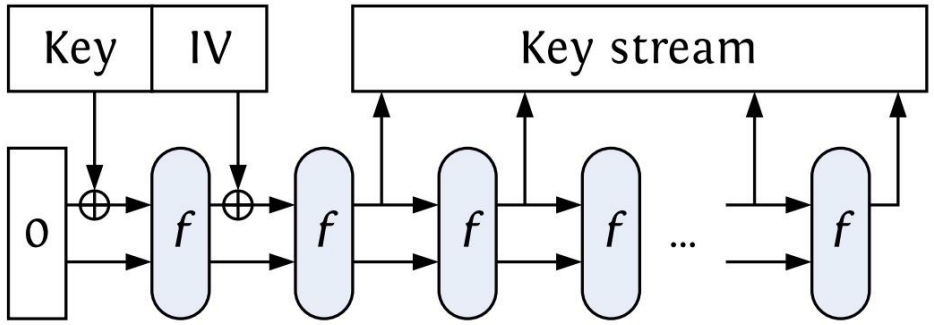
Source: J. Daemen's presentation at DIAC 2012
https://hyperelliptic.org/DIAC/

Permutation-based construction: sponge

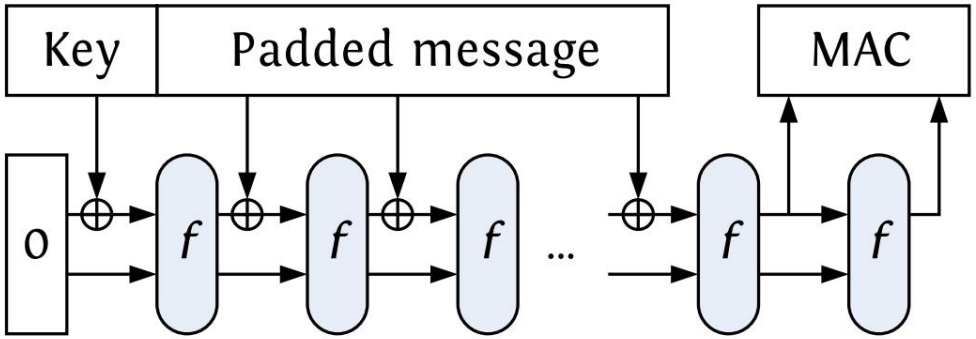
Very flexible design!



hash function



stream cipher



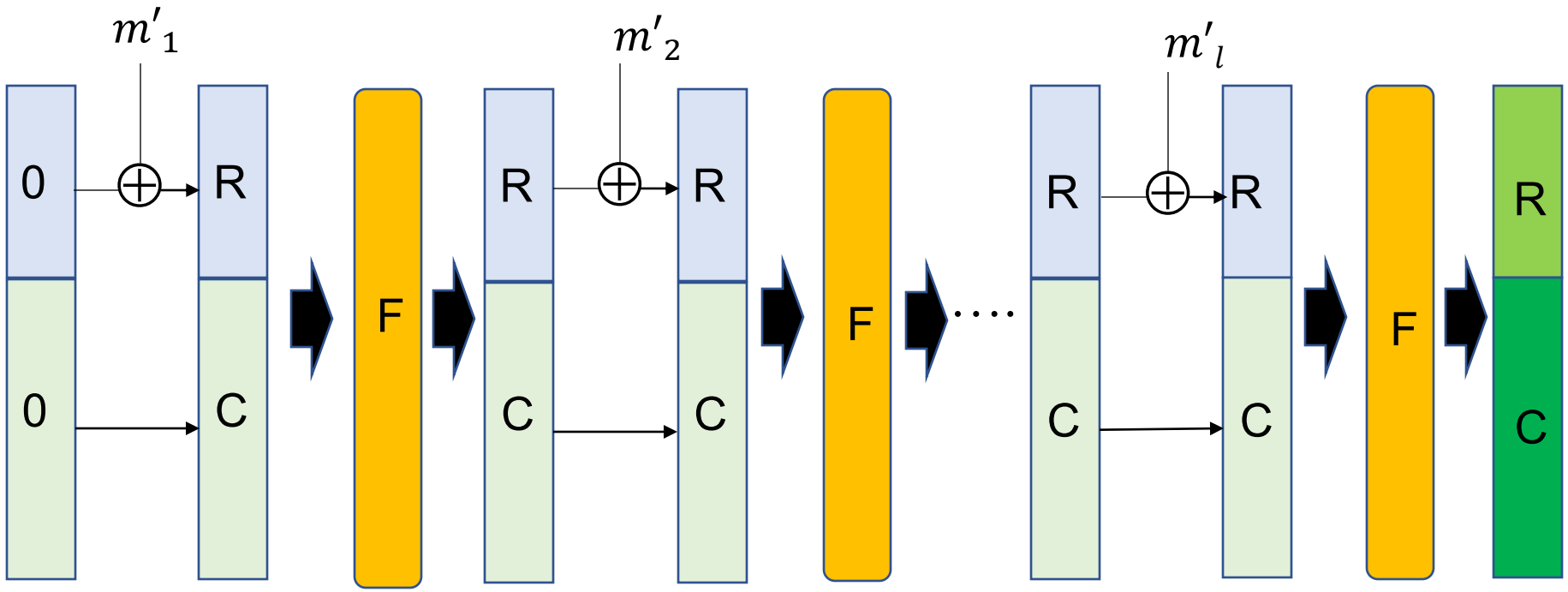
Message Authentication Code

Sponges are also particularly suitable for constructing Authenticated Encryption schemes

Source: J. Daemen's presentation at DIAC 2012 https://hyperelliptic.org/DIAC/

Sponge construction in SHA-3 – absorbing phase

Pad(m) = $m'_1\ m'_2\ \ldots\ m'_l$ (Merkle-Damgård)



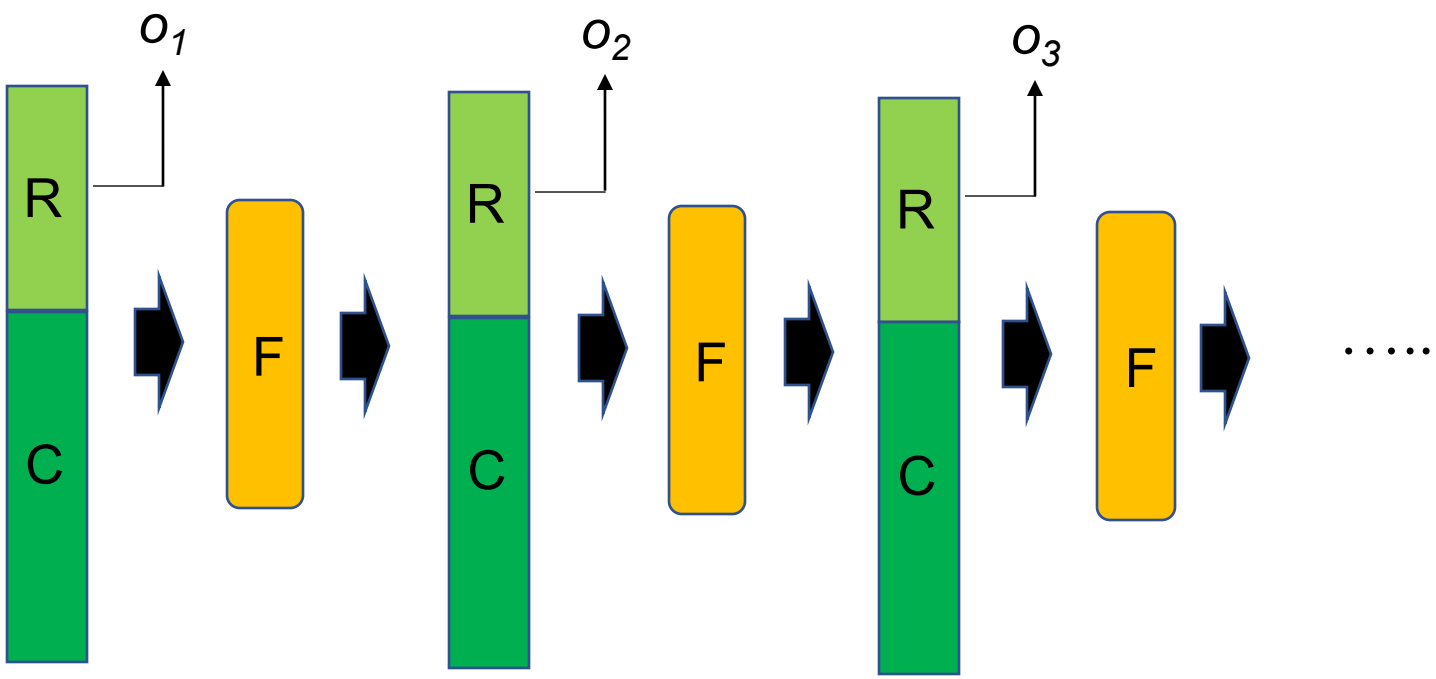
m'_i : padded message blocks, 1088 bits

R: outer state, 1088 bits

C: inner state, 512 bits

F: bit permutation mapping $\{0,1\}^{1600}$ to $\{0,1\}^{1600}$ (the Keccak permutation)

Sponge construction in SHA-3 – squeezing phase



SHA-3(m) = $o_1\ o_2\ o_3$...

(Some) applications of hash functions

Data fingerprinting

- Scenario 1
  - You want to protect the integrity of key system files (against hackers modifying them), by hashing the files and securely storing the hashes
  - The system regularly checks whether the hashes match
- Scenario 2
  - You want to distribute a program online, and users want to ensure its integrity before installation
  - You can then also distribute the hash via a (possibly low-bandwidth) authentic channel, and users can check that the hash matches

- **Security property?**
  - 2nd pre-image resistance

Password protection

- Scenario:
  - Authenticating systems, instead of storing *(username, password)*, store *(username, H(password))*
  - When attempting to log on, the system computes the hash of the password provided, and checks a match with stored (*username, hash*)
  - Note that any string *s* satisfying *H(s) = H(password)* will be considered as a valid password for the purpose of logging on to the system

- **Security property?**
  - Pre-image resistance

Commitment schemes

- You decided to run a competition, in which players need to guess the winner of the next World Cup
- Participants must guess and **commit** to that guess today
- Participants want to keep their choices private
- **First attempt**:
  - Ask them to submit their guesses as *H(winner)*
  - Then, once the World Cup ends, they can disclose their guess and prove it was correct!
- **Attack**:
  - Since there aren't that many countries, you could recover their guesses by running through all countries, computing the hash and comparing with their commitment

Commitment schemes

- You decided to run a competition, in which players need to guess the winner of the next World Cup
- Participants must guess and **commit** to that guess today
- Participants want to keep their choices private
- **Second attempt**:
  - Ask them to submit their guesses $H(winner \mid\mid r)$, where r is a k-bit random string
  - Then, once the World Cup ends, they can disclose their guess and the random string to prove that their country guess was correct!
  - Is this secure?

Homework

- Read Sections 6.1 and 6.2 of “Everyday Cryptography”
- Read Chapter 6 (p. 105-111) of “Serious Cryptography”